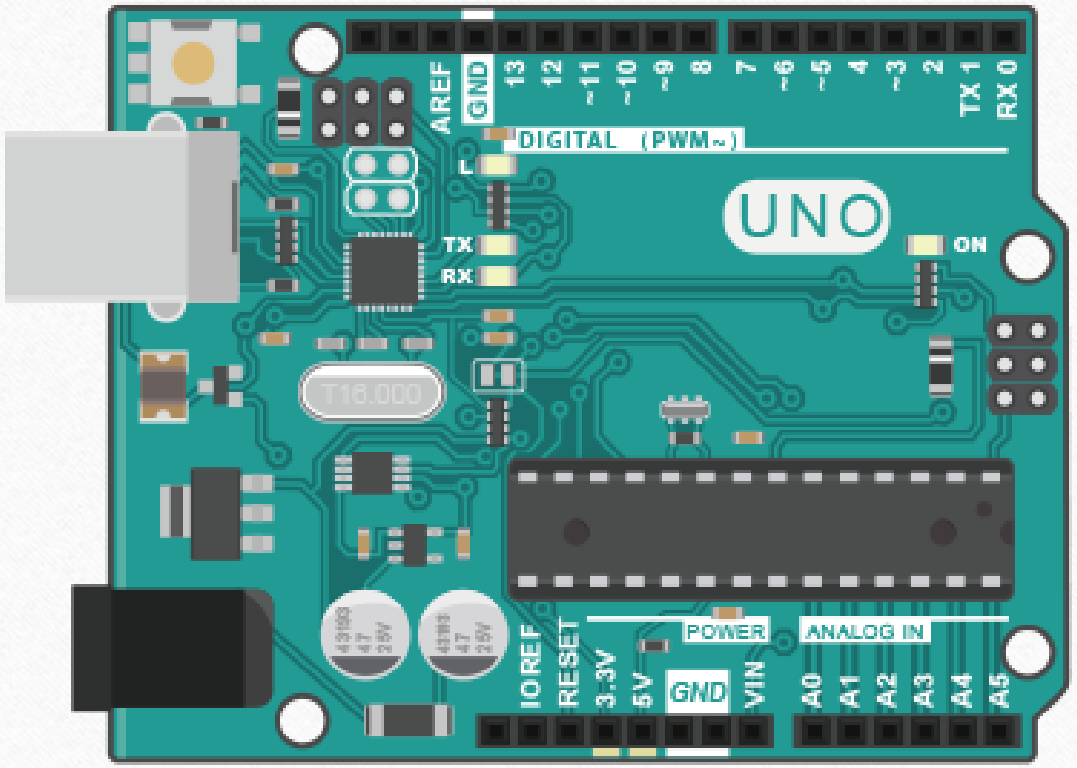
Simple Digital and Analog Input

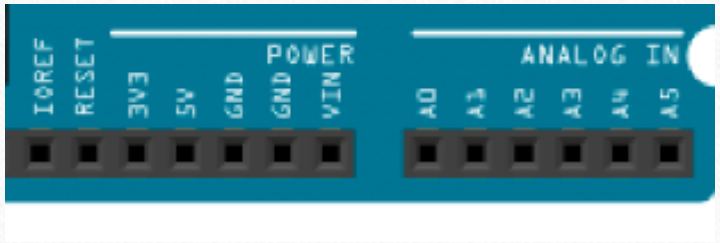
Chao-Chin Wu

Reference: M. Margolis, et al., Arduino Cookbook, 3rd Edition, Oreilly, 2020

Arduino Uno


AREF
GND
13
12
~11
~10
~9
8
7
~6
~5
4
~3
2
TX 1
RX 0
DIGITAL (PWM~)
L
TX
RX
UNO
ON
POWER
ANALOG IN
IOREF
RESET
3.3V
5V
GND
VIN
A0
A1
A2
A3
A4
A5

- Digital input pins sense the presence and absence of voltage on a pin
- Analog input pins measure a range of voltages on a pin.



- 14 digital pins
  - numbered 0 to 13
  - pins 0 and 1 (marked RX and TX) are used for the USB serial connection
    - should be avoided for other uses

- Arduino has logical names that can be used to reference many of the pins.
- you should avoid using the numeric pin number and use these constants instead.

Constant	Pin	Constant	Pin
A0	Analog input 0	LED_BUILTIN	Onboard LED
A1	Analog input 1	SDA	I2C Data
A2	Analog input 2	SCL	I2C Clock
A3	Analog input 3	SS	SPI Select
A4	Analog input 4	MOSI	SPI Input
A5	Analog input 5	MISO	SPI Output
		SCK	SPI Clock

Introduction

- As with the standard board, you can use analog pins as digital pins.
- There are 16 digital pins on the Arduino board. They can be used as inputs or outputs. They operate at 5V and have a maximum current draw of 40mA.

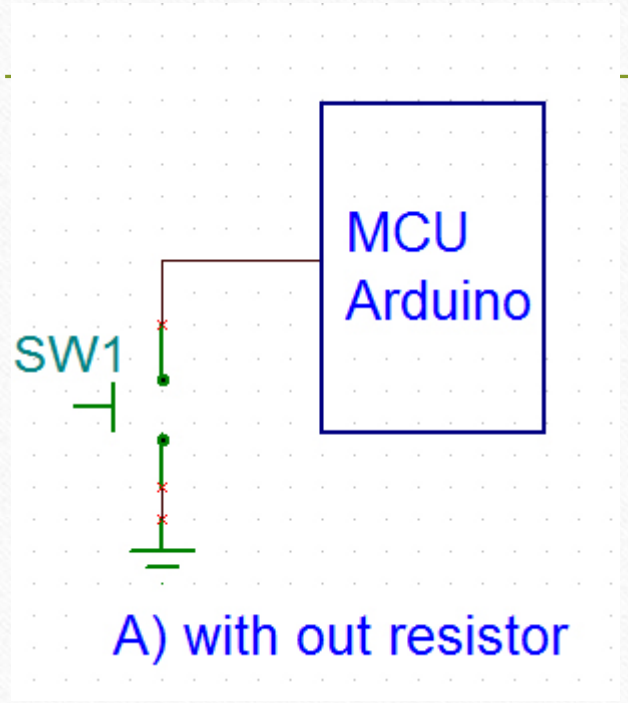
- Each pin can provide or receive a maximum of **40 mA**…”
- And the whole chip should not draw current above a absolute maximum of 200 mA
- the maximum output current an Arduino's digital pin can supply is **40mA (or 20mA continuous current)**.
  - You cannot use it to drive more than two 20mA LEDs in parallel.
  - Likewise, the digital pin outputs 5V.

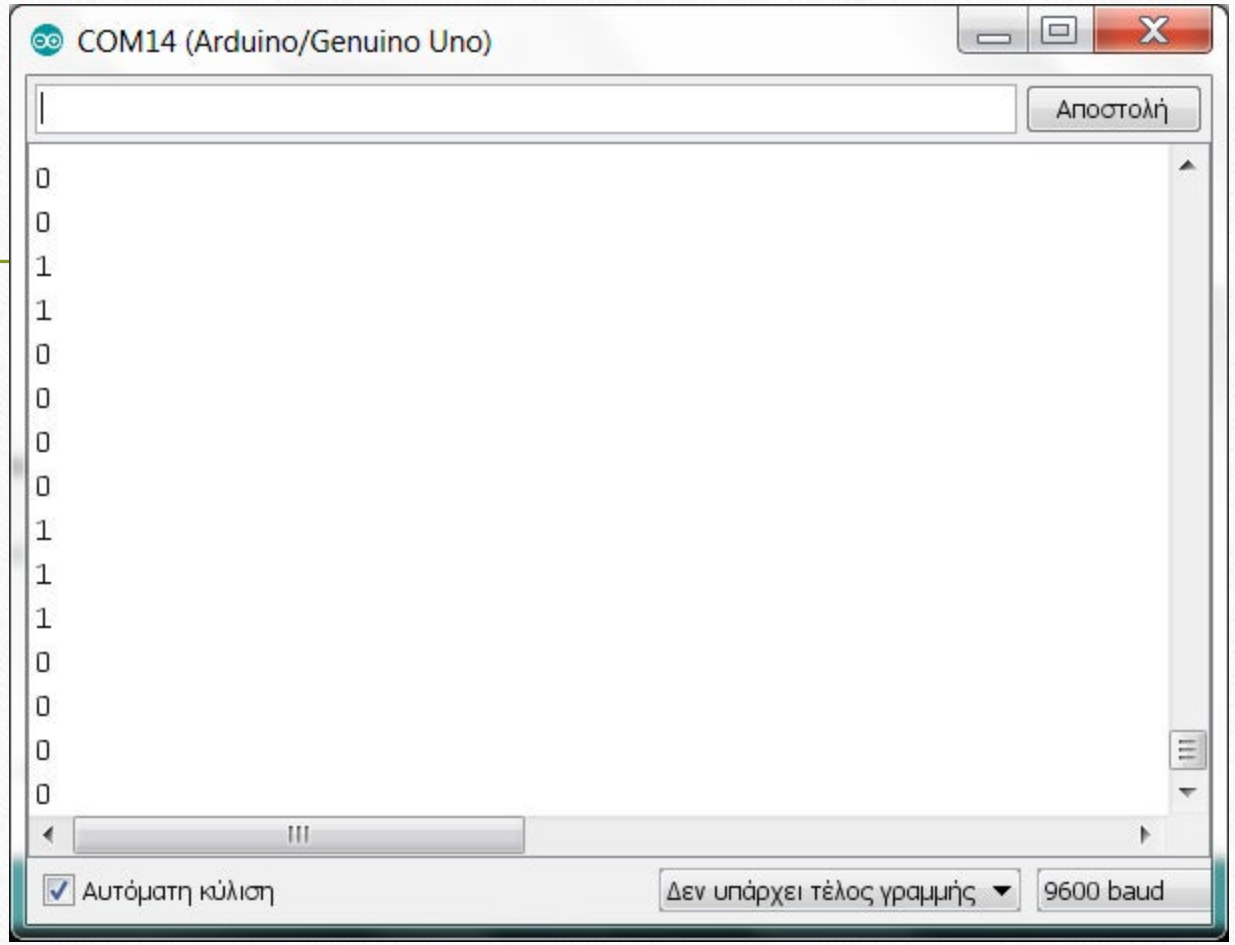
- Digital pins sometimes use internal or external resistors to force the input pin to stay in a known state when the input is not engaged.
  - Without such a resistors, the pin's value would be in a state known as *floating*
    - digitalRead might return HIGH at first, but then would return LOW milliseconds later, regardless of the whether the input is engaged (such as when a button is pressed)
- A pull-up resistor is so named because the voltage is "pulled-up" to the logic level (5V or 3.3V) of the board.
  - When you press the button in a pull-up configuration, digitalRead will return LOW.
  - At all other times, it returns HIGH.

- A pull-down resistor pulls the pin down to 0 volts.
- Although 10K ohms is a commonly used value for a pull-up or pull-down resistor, anything between 4.7K and 20K or more will work.
- Arduino boards have internal pull-up resistors that you can activate when you use the INPUT_PULLUP mode with pinMode

- Unlike a digital value, analog values are continuously variable.
- Many sensors provide information by varying the voltage to correspond to the sensor measurement.
- analogRead
  - get a value proportional to the voltage it sees on one of its analog pins.
  - 0 ~ 1023 (0V to 5V or 3.3V)

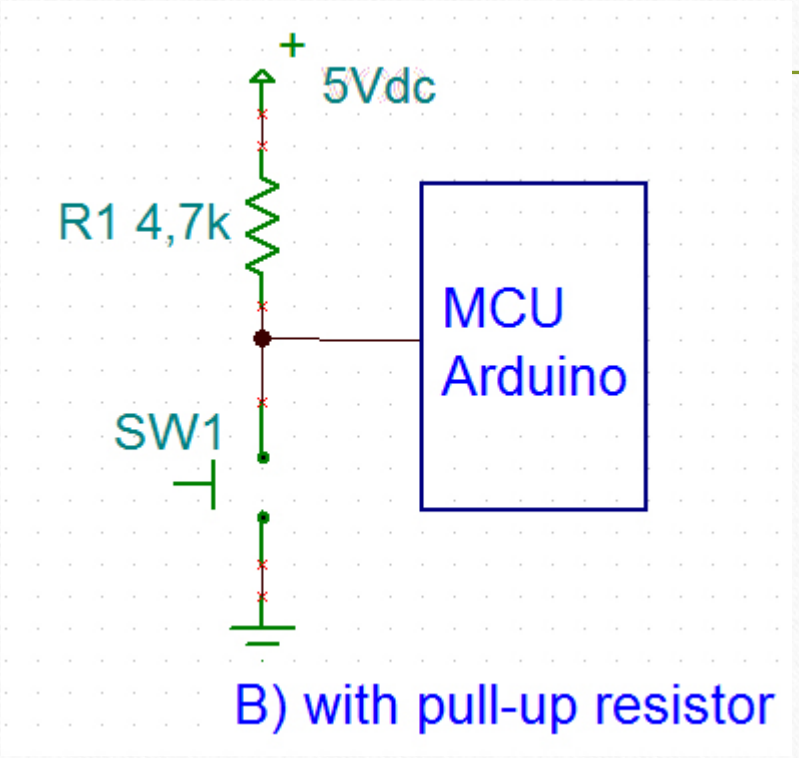
- A potentiometer (pot for short, also called a variable resistor) can be used to vary the voltage on a pin.
  - When choosing a potentiometer, a value of 10K is the best option for connecting to analog pins.

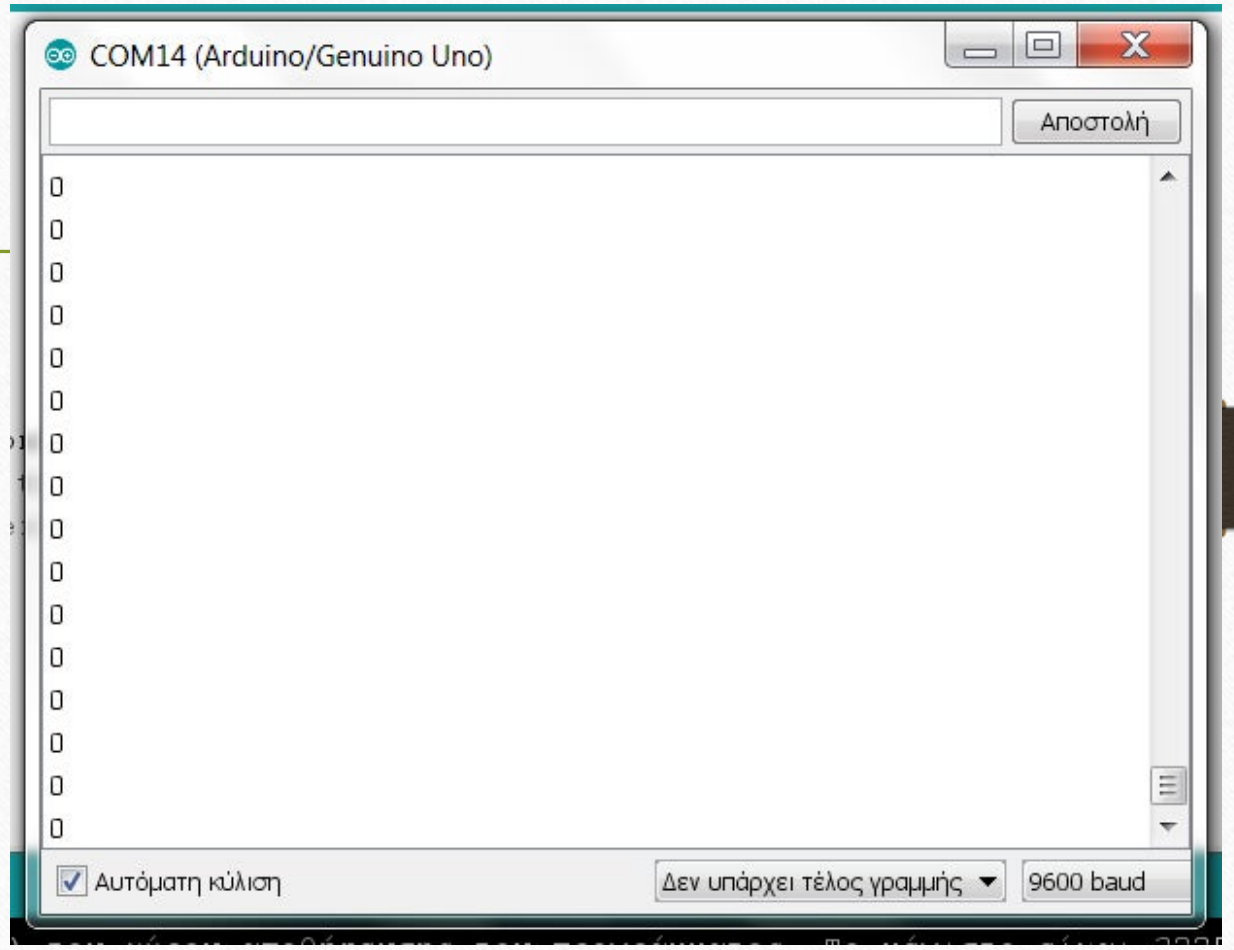




Source: https://www.instructables.com/Understanding-the-Pull-up-Resistor-With-Arduino/

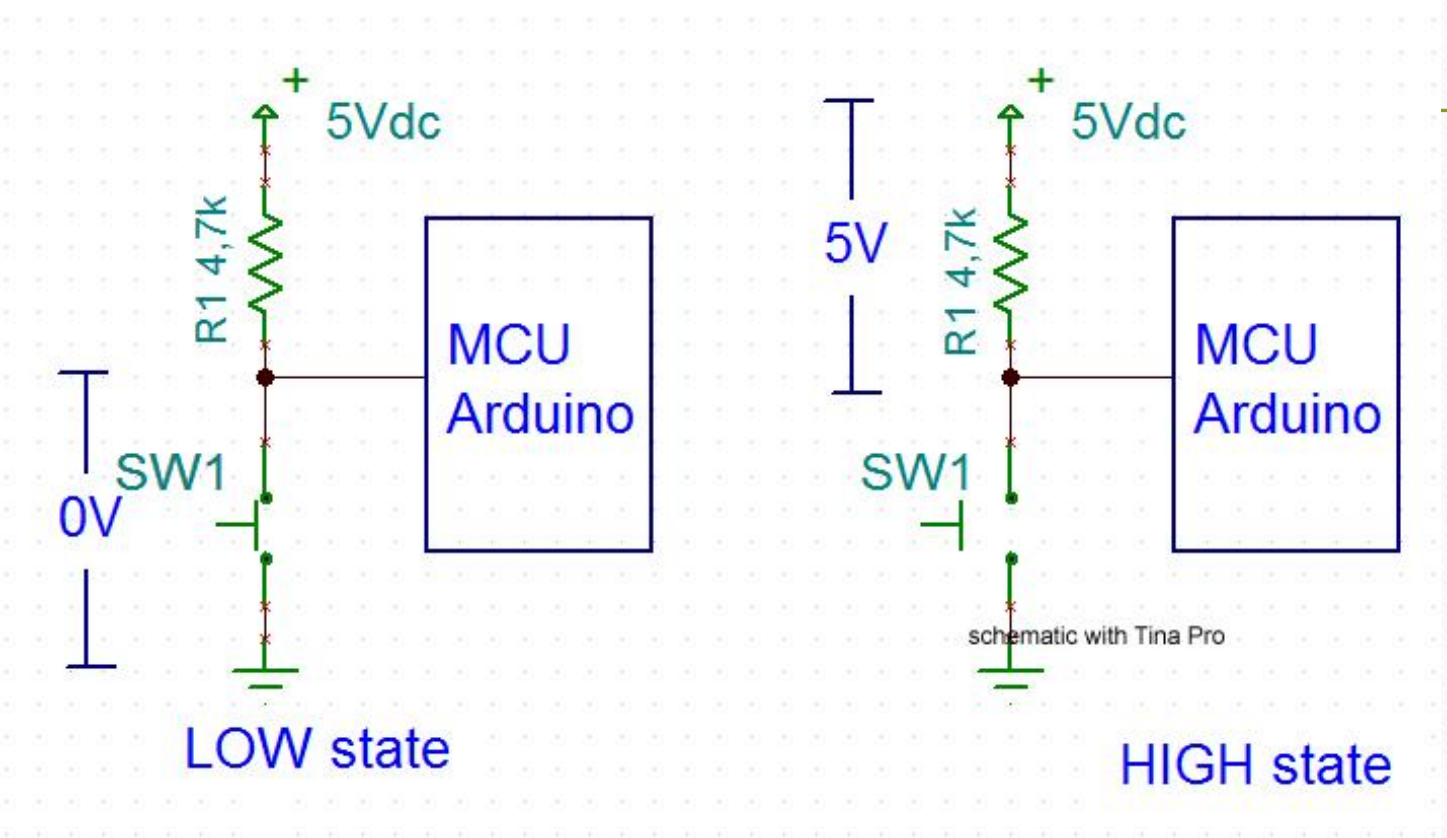
Pull-Up Resistor





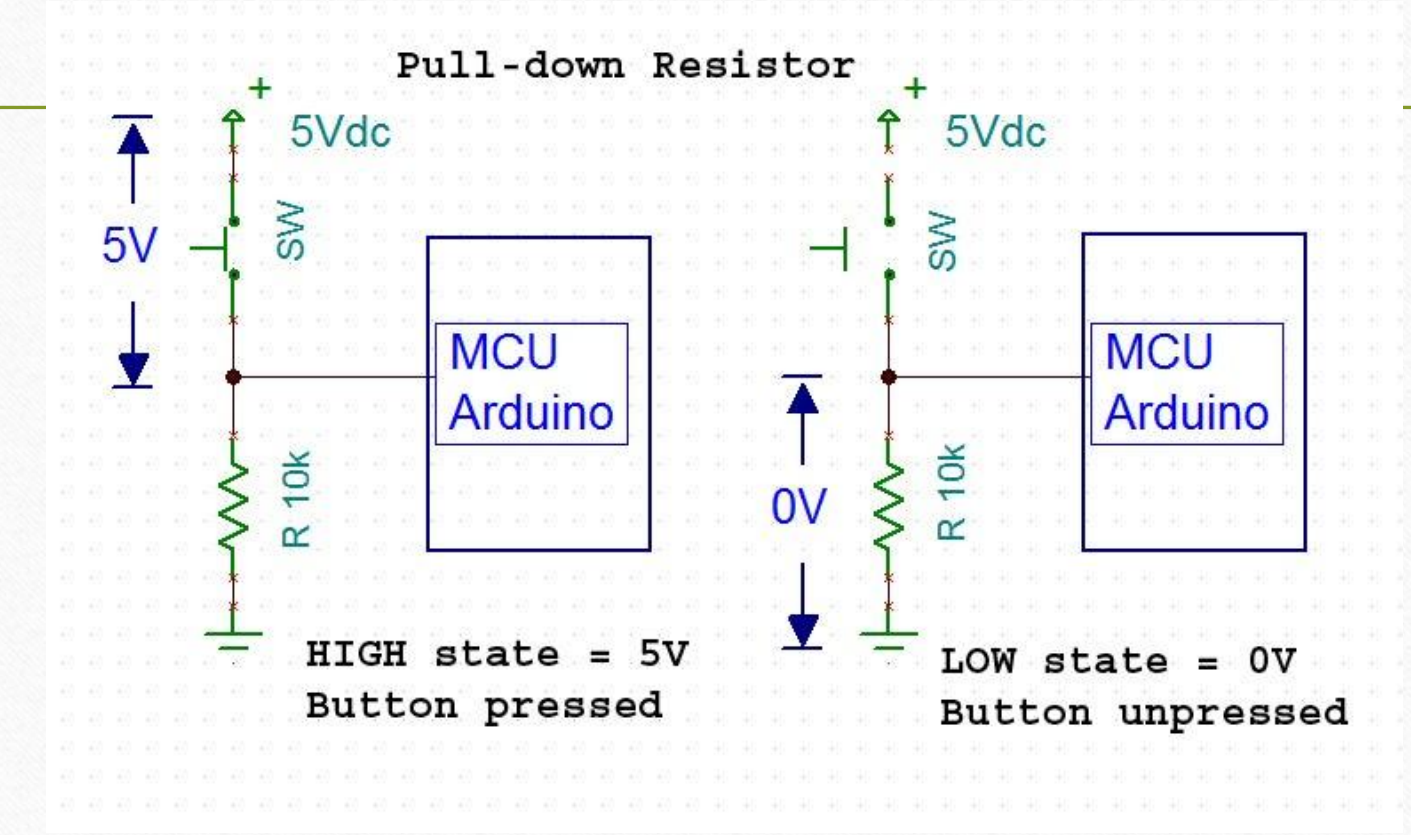
Source: https://www.instructables.com/Understanding-the-Pull-up-Resistor-With-Arduino/

Pull-Up Resistor



Source: https://www.instructables.com/Understanding-the-Pull-up-Resistor-With-Arduino/

Pull-Down Resistor



Source: https://www.instructables.com/Understanding-the-Pull-up-Resistor-With-Arduino/

Pull-Up and Pull-Down Resistors

- pull-up resistor 上拉電阻
  - https://www.youtube.com/watch?v=g4QH60aSLWA
- pull-down resistor 下拉電阻
  - https://www.youtube.com/watch?v=o8_KquaC7Mk

Reference:爱上半导体

Pin Mode Setting

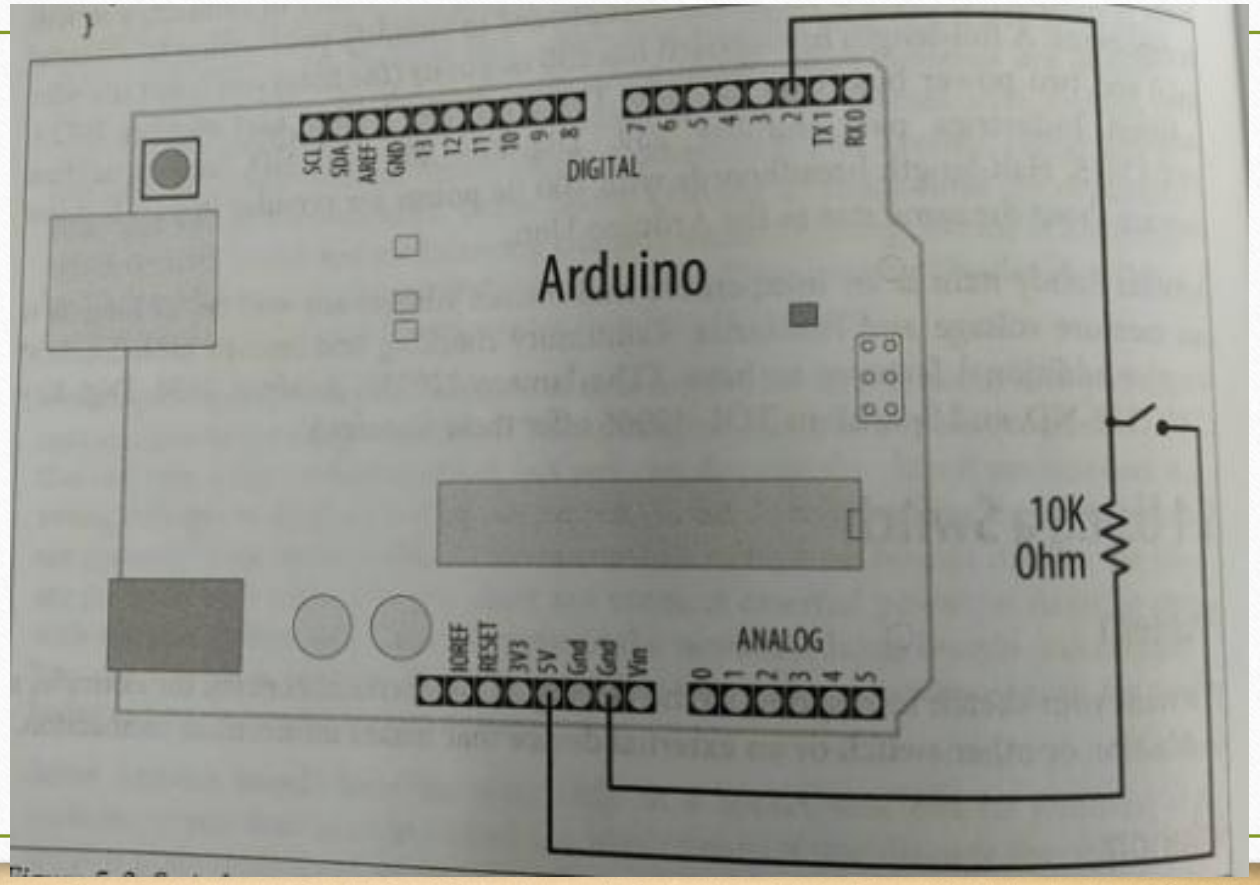
- pinMode(pin, INPUT)
  - configure a pin for reading input
- pinMode(pin, OUTPUT)
  - configure a pin for sending output

Read and Write for Digital Pins

- digitalRead function
  - detect digital input
  - tells your sketch if a voltage on a pin is HIGH or LOW
    - HIGH is between 3 and 5 volts for the Uno
    - LOW is 0 volts
- digitalWrite function
  - Write a HIGH or a LOW value to a digital pin.

Using a Switch
Lights an LED when a switch is pressed

Switch connected using pull-down resistor

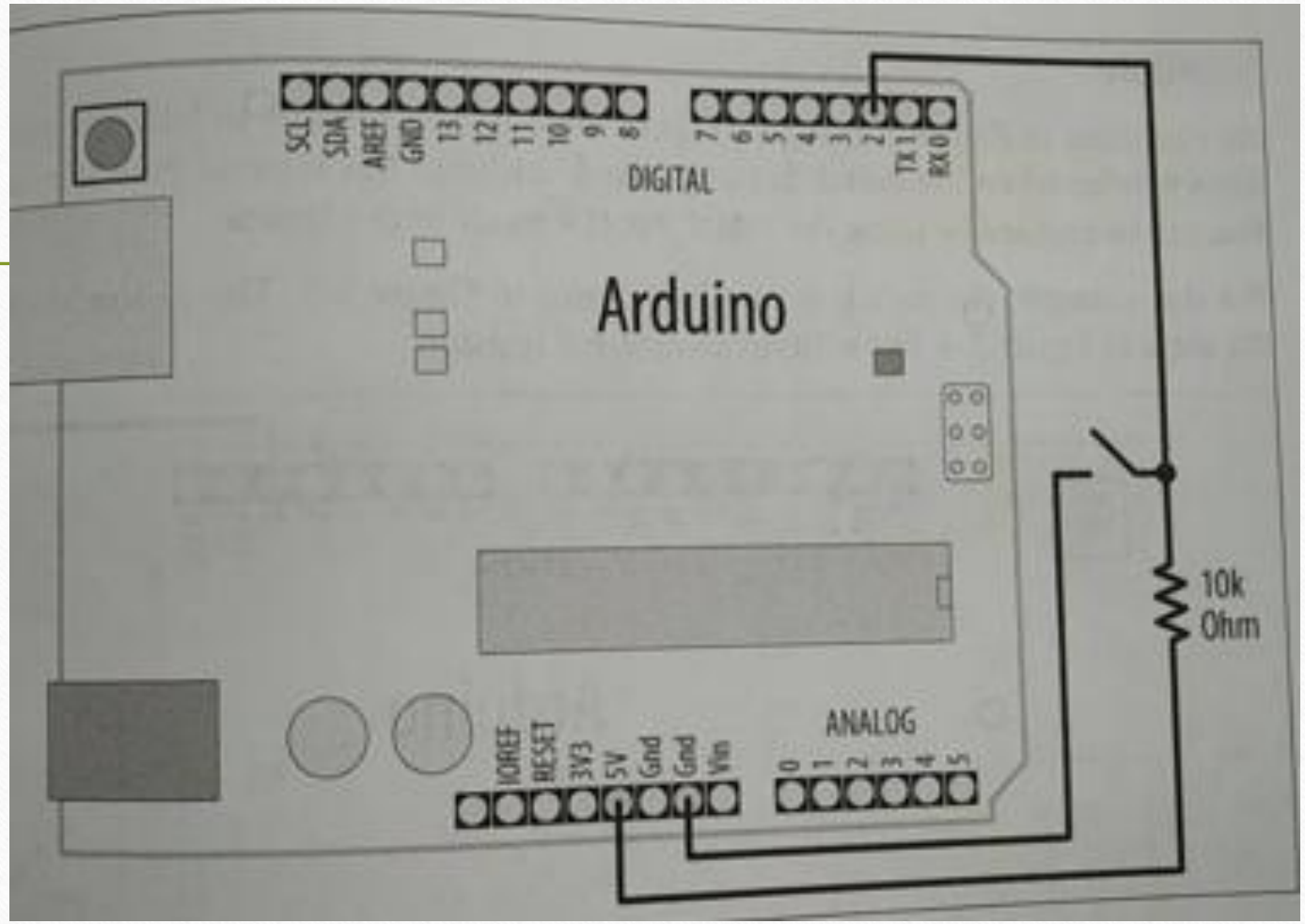


Using a Switch
Lights an LED when a switch is pressed

```
1   /*
2      Pushbutton sketch
3      a switch connected to pin 2 lights the built-in LED
4   */
5
6   const int inputPin = 2;           // choose the input pin (for a p
7
8   void setup() {
9     pinMode(LED_BUILTIN, OUTPUT);   // declare LED as output
10    pinMode(inputPin, INPUT);       // declare pushbutton as input
11  }
```

```
13  void loop(){
14    int val = digitalRead(inputPin);  // read input value
15    if (val == HIGH)                  // check if the input is HIGH
16    {
17      digitalWrite(LED_BUILTIN, HIGH); // turn LED on if switch is pressed
18    }
19    else
20    {
21      digitalWrite(LED_BUILTIN, LOW);  // turn LED off
22    }
23  }
```

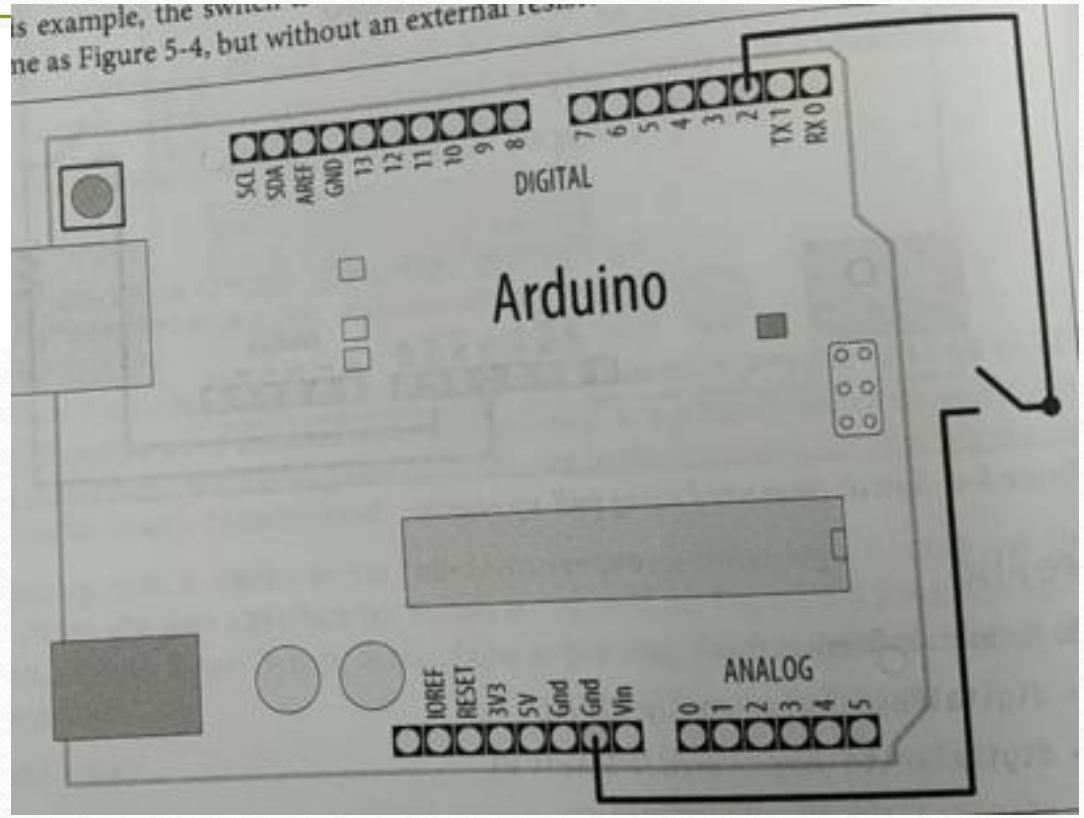
- Switch connected using pull-up resistor



```
if (val == HIGH)                    // check if the input is HIGH
{
  digitalWrite(LED_BUILTIN, LOW);  // turn LED OFF
}
else
{
  digitalWrite(LED_BUILTIN, HIGH); // turn LED ON
}
```

Using a Switch Without External Resistors

```
void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
  pinMode(inputPin, INPUT_PULLUP); //
}
```



Reliable Detect (Debounce) When a Switch is Pressed

- Contact bounce
  - produces spurious signals at the moment the switch contacts close or open
- Avoid false readings due to contact bounce
  - Debouncing

```
1   /*
2    * Debounce sketch
3    * a switch connected to pin 2 lights the built-in LED
4    * debounce logic prevents misreading of the switch state
5    */
6
7   const int inputPin = 2;         // the number of the input pin
8   const int debounceDelay = 10;   // iterations to wait until pin is stable
9   bool last_button_state = LOW;   // Last state of the button
10  int ledState = LOW;             // On or off (HIGH or LOW)
```

```
18      previousState = digitalRead(pin);          // store switch state
19      for(int counter=0; counter < debounceDelay; counter++)
20                                                 //設定連續幾次檢查，狀態都不變
21      {
22          delay(1);                  // wait for 1 millisecond
23          state = digitalRead(pin);  // read the pin
24          if( state != previousState)
25          {
26              counter = 0; // reset the counter if the state changes
27              previousState = state;  // and save the current state
28          }
29      }
30      // here when the switch state has been stable longer than the debounce period
31      return state;
    }
```

```
33  void setup()
34  {
35    pinMode(inputPin, INPUT);
36    pinMode(LED_BUILTIN, OUTPUT);
37  }
38
39  void loop()
40  {
41    bool button_state = debounce(inputPin);
42
43    // If the button state changed and the button was pressed
44    if (last_button_state != button_state && button_state == HIGH) {
45      // Toggle the LED
46      ledState = !ledState;
47      digitalWrite(LED_BUILTIN, ledState);
48    }
49    last_button_state = button_state;
50  }
```

- A potential disadvantage of this method for some applications
  - from the time the debounce function is called, everything waits until the switch is stable.
- Your sketch may need to be attending to other things while waiting for your switch to stable.
- You can use the code shown in the following to overcome this problem.

Determining How Long a Switch Is Pressed

- Example: setting a countdown timer
  - Pressing a switch sets the timer by incrementing the timer count.
  - Releasing the switch starts the countdown.
  - The code debounces the switch and accelerates the rate at which the counter increases when the switch is initially pressed (after debouncing).
    - holding the switch for more than one second increases the rate by four
    - holding the switch for four seconds increases the rate by 10.
    - Releasing the switch starts the countdown, and when the count reaches zero, lights an LED.

```
const int ledPin = LED_BUILTIN;          // the number of the output pin
const int inPin = 2;                     // the number of the input pin

const int debounceTime = 20;             // the time in milliseconds required
                                         // for the switch to be stable
const int fastIncrement = 1000;          // increment faster after this many
                                         // milliseconds
const int veryFastIncrement = 4000;      // and increment even faster after
                                         // this many milliseconds
int count = 0;                           // count decrements every tenth of a
                                         // second until reaches 0
void setup()
{
  pinMode(inPin, INPUT_PULLUP);
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}
```

```
27  void loop()
28  {
29    int duration = switchTime();
30    if(duration > veryFastIncrement)
31    {
32      count = count + 10;
33    }
34    else if (duration > fastIncrement)
35    {
36      count = count + 4;
37    }
38    else if (duration > debounceTime)
39    {
40      count = count + 1;
41    }
```

```
42    else
43    {
44      // switch not pressed so service the timer
45      if( count == 0)
46      {
47        digitalWrite(ledPin, HIGH);  // turn the LED on if the count is 0
48      }
49      else
50      {
51        digitalWrite(ledPin, LOW);   // turn the LED off if the count is not 0
52        count = count - 1;           // and decrement the count
53      }
54    }
55    Serial.println(count);
56    delay(100);
57  }
```

```
// return the time in milliseconds that the switch has been in pressed (LOW)
long switchTime()
{
  // these variables are static - see Discussion for an explanation
  static unsigned long startTime = 0; // when switch state change was detected
  static bool state;                  // the current state of the switch

  if(digitalRead(inPin) != state) // check to see if the switch has changed state
  {
    state = ! state;       // yes, invert the state
    startTime = millis();  // store the time
  }
  if(state == LOW)
  {
    return millis() - startTime;   // switch pushed, return time in milliseconds
  }
  else
  {
    return 0; // return 0 if the switch is not pushed (in the HIGH state);
  }
}
```

Analog Input Pins

- **A/D converter**
  - The ATmega controllers used for the Arduino contain an onboard 6 channel (8 channels on the Mini and Nano, 16 on the Mega) analog-to-digital (A/D) converter.
  - The converter has 10 bit resolution, returning integers from 0 to 1023.
  - While the main function of the analog pins for most Arduino users is to read analog sensors, the analog pins also have all the functionality of general purpose input/output (GPIO) pins (the same as digital pins 0 - 13).
  - Consequently, if a user needs more general purpose input output pins, and all the analog pins are not in use, the analog pins may be used for GPIO.

Analog Input Pins

- **Pin mapping**
  - The analog pins can be used identically to the digital pins, using the aliases A0 (for analog input 0), A1, etc.
  - For example, the code would look like this to set analog pin 0 to an output, and to set it HIGH:

```
pinMode(A0, OUTPUT);
digitalWrite(A0, HIGH);
```

Analog Input Pins

- **Pull-up resistors**
  - The analog pins also have pull-up resistors, which work identically to pull-up resistors on the digital pins.
  - They are enabled by issuing a command such as

```
pinMode(A0, INPUT_PULLUP);    // set pull-up on analog pin 0
```

  - Be aware however that turning on a pull-up will affect the values reported by analogRead().

Analog Input Pins

- The analogRead command will not work correctly if a pin has been previously set to an output, so if this is the case, set it back to an input before using analogRead.
- Similarly if the pin has been set to HIGH as an output, the pull-up resistor will be set, when switched back to an input.

Reading Analog Values

- reads the voltage on an analog pin (A0) and flashes an LED at a proportional rate to the value returned from a potentiometer

```
const int potPin = A0;           // select the input pin for the potentiometer
const int ledPin = LED_BUILTIN; // select the pin for the LED
int val = 0;  // variable to store the value coming from the sensor

void setup()
{
  pinMode(ledPin, OUTPUT);  // declare the ledPin as an OUTPUT
}
```

// The analogRead() function takes care of setting up the pin. You don't need to set it as input.

```
void loop() {
  val = analogRead(potPin);   // read the voltage on the pot
  digitalWrite(ledPin, HIGH); // turn the ledPin on
  delay(val);                 // blink rate set by pot value (in milliseconds)
  digitalWrite(ledPin, LOW);  // turn the ledPin off
  delay(val);                 // turn led off for same period as it was turned on
}
```

//value: 0 ~ 1023

- You can use the Arduino map function to scale values to the range you want.

```
percent = map(val,0,1023,0,100); // percent will range from 0 to 100.
```

  - *map uses integer math (Any fraction element is truncated, not rounded)*

Reading More than Six Analog Inputs

- Use a multiplexer chip to select and connect multiple voltage sources to one analog input.